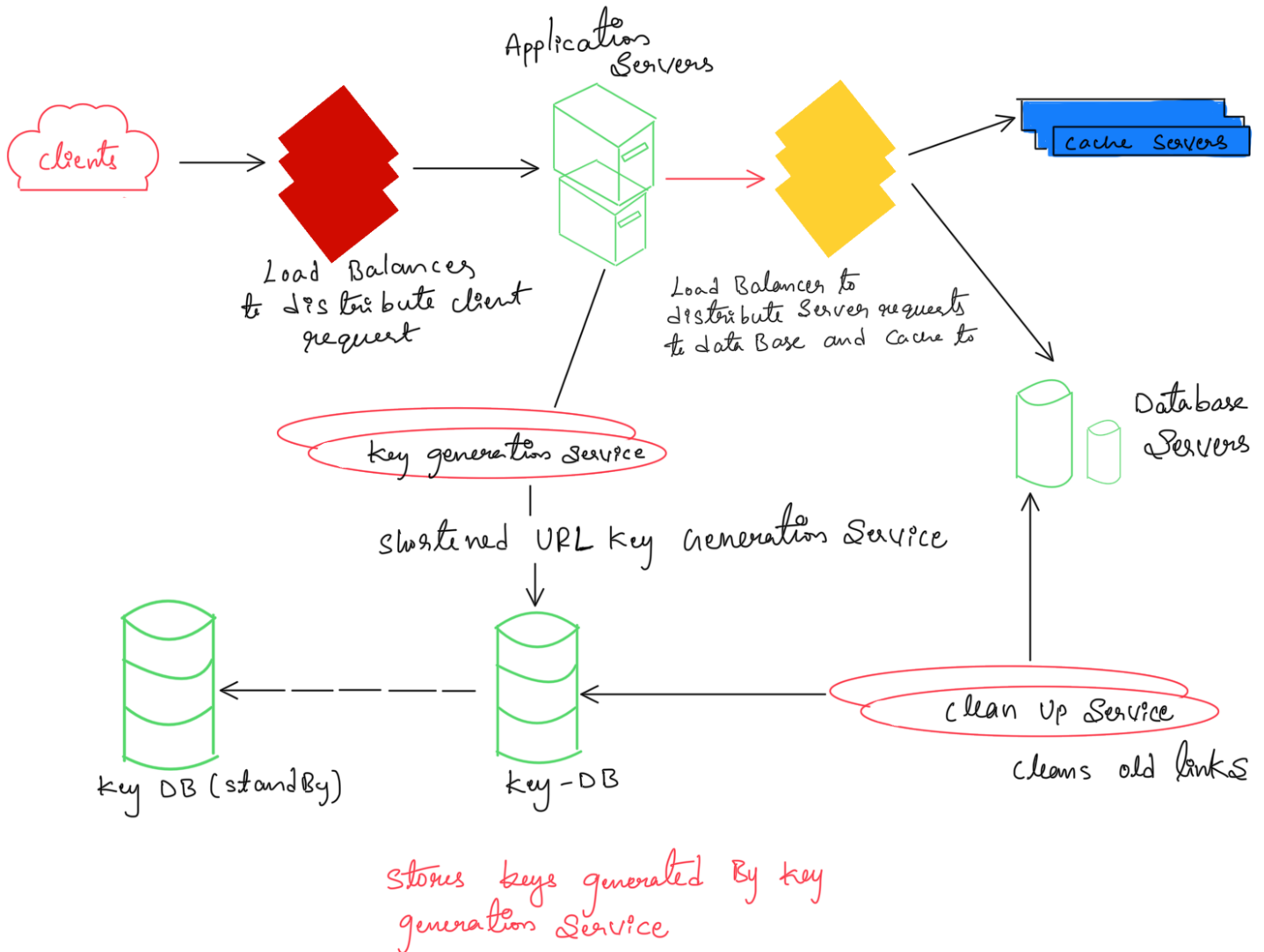


High level Design of a URL-Shortening Service



- **User Interface/clients**
Users enter a long URL via Web form or API and receive a shortened link.
- **Application Server**
generates Unique short keys, stores mapping in the database, handles redirects, and tracks clicks.
- **Load Balancer**
Distributes traffic across Servers at three points
 - clients - Application Servers

- Application Servers - Database
- Application Servers - Cache

• Database

Stores URL - key mappings, ensures uniqueness, and scales with NoSQL solution like MongoDB or Cassandra

• Caching

Speeds up reads with in-memory caches (Redis/Memcached) for frequently accessed URLs.

• Cleanup Service

Removes Expired or Un Used data from storage

• Redirection

Short key lookup retrieves the original URL and redirects using HTTP 301.

• Analytics

Track clicks, referrers, browsers, and device for User Insights

• Security

Prevents abuse (phishing/malware links, DDoS brute force) using firewalls, rate-limiting, and authentication.

Database Design

Let us explore some of the choices for System Design of Databases of URL Shortener.

- We can use RDBMS which uses ACID properties But you will be facing the scalability issue with relational databases.
- Now if you think you can use sharding and resolve the scalability issue in RDBMS then that will increase the complexity of the system
- There are 30M active users so there will be conversions

and a lot of short URL resolution and redirects

- Read and Write will be heavy for these 30M users so scaling the RDBMS using shard will increase the complexity of the design.

you may have to use consistent hashing to balance the traffic and DB queries in the case of RDBMS and which is a complicated process. So to handle this amount of huge traffic on our system relational databases are not fit and it won't be a good decision to scale the RDBMS

So let's take a look at NoSQL Database:

- The only problem with using the NoSQL database is its eventual consistency.
- We write something and it takes some time to replicate to a different node but our system needs high availability and NoSQL fits this requirement.
- NoSQL can easily handle the 30M of active users and it is easy to scale. We just need to keep adding the nodes when we want to expand the storage.

Caching and Load Balancing in URL Shortening service

In a URL shortening service, caching and load balancing are essential for managing high demand and optimizing response times. The service could greatly benefit from read-through caching or write-through caching mechanisms.

- A read-through cache automatically loads data into the cache when a miss occurs
- While a write-through cache updates the cache whenever the database is updated.

- **Caching**

Use a read-through cache (Redis/Memcached) to quickly serve frequently accessed shortened URLs.

- **Load Balancing**

- **Round Robin:** Evenly spreads traffic, simple to scale.
- **Least Connections:** Best when request processing times vary, balances load more efficiently.